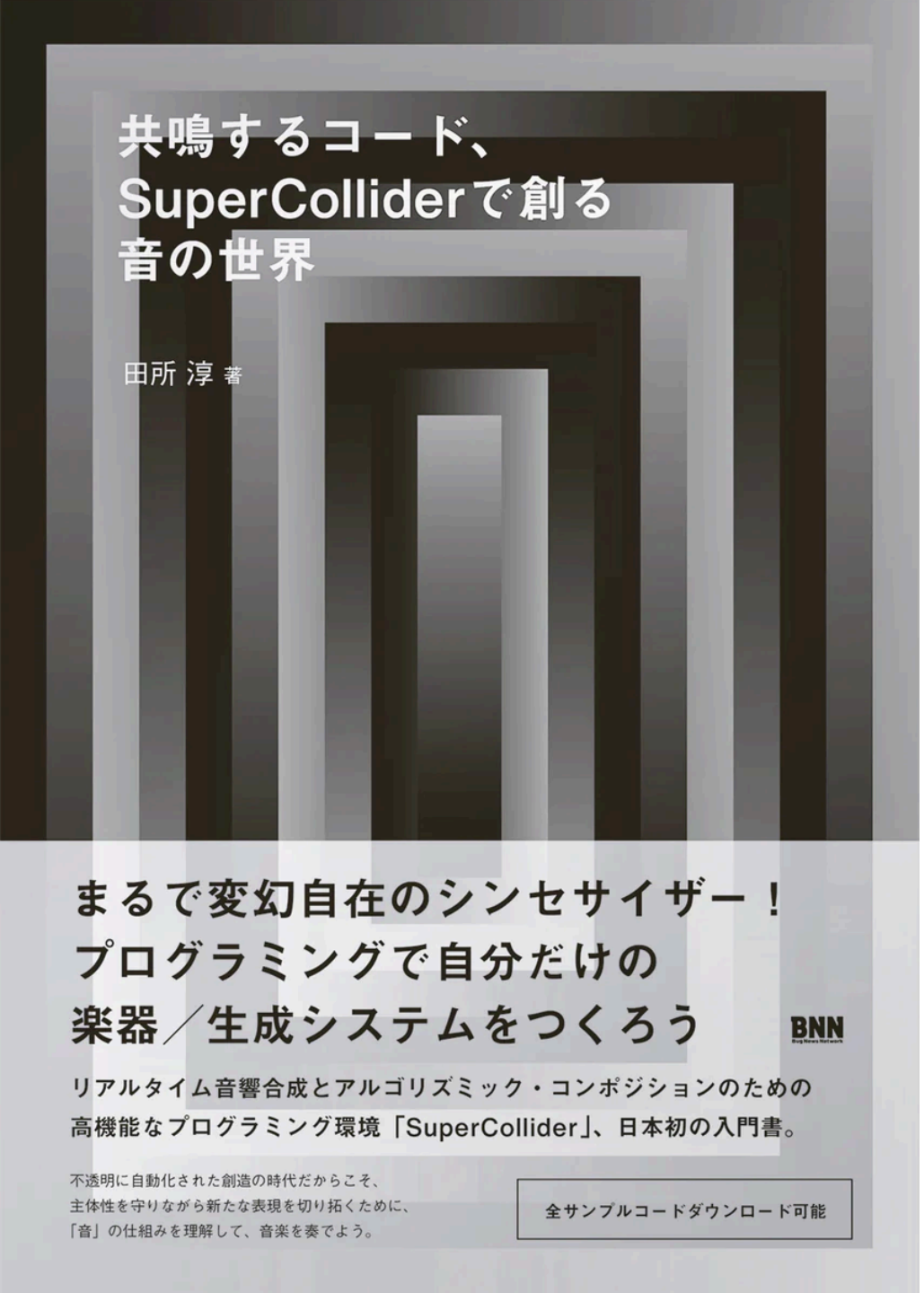
AMCJワークショップ

「共鳴するコード、SuperColliderで創る音の世界」実践とその先へ

2026年3月14日

田所淳



本日のワークショップの内容

書籍「共鳴するコード、SuperColliderで創る音の世界」のさらに先を紹介!

もちろん、SuperColliderの初歩からカバーしていきます。

環境設定

必要なアプリケーションをまずはインストールしていきましょう!

macOS / Windows どちらでもOK

- SuperCollider
- VScode
- Google Chrome

環境設定

本日のワークショップでは、コード生成のAIのアカウントがあるとなお便利です!

- Claude Code
- Github Copilot
- Gemini Code Assist
- ChatGPT codex

File Session Edit View Language Server Help

Chapter06.scd

```
// 第6章

/*
 * list-6_01
 */
// サーバーを起動
s.boot;

// サウンドファイルをバッファーに読み込み
// SuperCollierに付属された「a11wlk01.wav」
b = Buffer.read(s, Platform.resourceDir +/+ "sounds/a11wlk01.wav");

// b.plotでバッファの波形を表示
b.plot;

// b.playでバッファを単純に再生
b.play;

// 使い終わったバッファはメモリを解放する
b.free;

/*
 * list-6_02
 */
(
b = [
  // サウンドファイルを3つ読み込み
  Buffer.read(s, Platform.resourceDir ++ "/sounds/a11wlk01-44_1.aiff"),
  Buffer.read(s, Platform.resourceDir ++ "/sounds/a11wlk01.wav"),
  Buffer.read(s, Platform.resourceDir ++ "/sounds/SinedPink.aiff")
];
)

// 配列の0番目のバッファを再生
b[0].play;

// 配列の1番目のバッファを再生
b[1].play;

// 配列の2番目のバッファを再生
b[2].play;
```

SuperCollider IDEの基本をおさらい

画面を操作しながら解説していきます。

Help browser　Home　Find in page...

SuperCollider　Browse　Search　Indexes ▼　Table Of Contents ▼

SuperCollider 3.14.1

Documentation home

NOTE: News in SuperCollider version 3.14

An introductory overview

SuperCollider is a cross-platform environment for audio synthesis and algorithmic composition used by musicians, artists and researchers working with sound. It can be installed and run on Linux, macOS and Windows, as well as microcomputers such as Raspberry Pi or Bela. *SuperCollider* was originally developed by James McCartney and is now distributed as Free Software under the GNU General Public License. It is maintained and developed by an active and enthusiastic community.

SuperCollider consists of three main components:

- **scsynth — A real-time audio engine implemented as a server.**
  The default server is scsynth. SuperCollider also includes an alternative implementation of scsynth called supernova which has multi-threading support to fully exploit the potential of multi-core CPUs.
- **sclang — A text-based programming language and interpreter that acts as a client to the server.**
  sclang controls audio synthesis processes on the server and offers an interactive

Post window　Auto Scroll　Clear

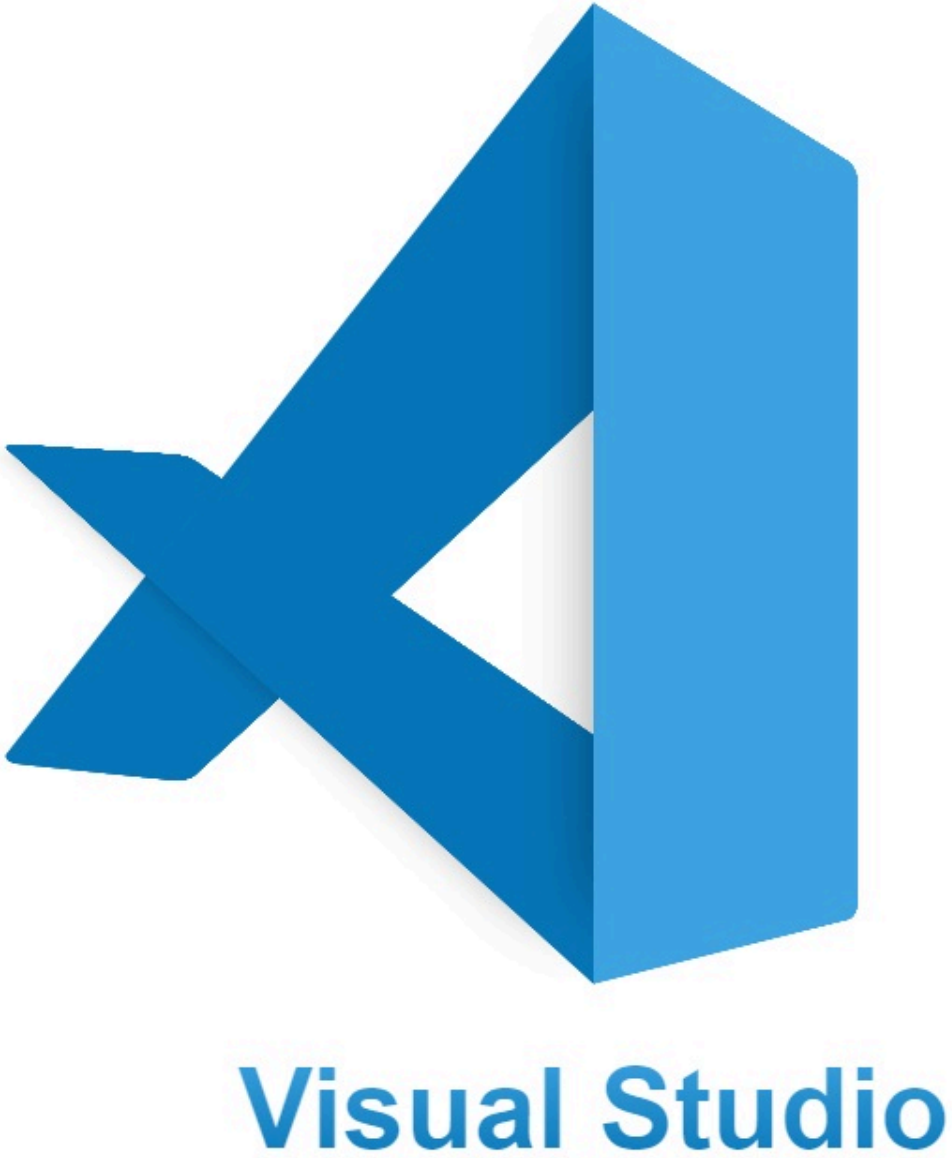
```
.git (0) 808 (6) 808bd (25) 808cy (25) 808hc (5) 808ht (5) 808lc
... file reading complete. Required 450 MB of memory.



162 existing sample banks:
.git (0) 20010320-Samples (23) 20020506 (0) adhan (4) ae (5) alib
replacing 'jungbass' (20)
jungbass (20) kon (17) kore (3) laurie (3) lm-2 (29) lowd (1) lum
replacing 'rs' (1)
rs (1) sbt (3) sfh (33) spiano (3) strav (1) sup (6) svr (2) sync
... file reading complete. Required 2467 MB of memory.

SuperDirt: listening on port 57120
```

本日の「その先」：1
VScodeからSuperColliderを使う

VScodeからSuperColliderを使う

SC IDEは便利ですが、VScodeを使用するとさらに様々なメリットが

- コード補完
- シンタックスハイライト
- Git連携
- **AIコード生成ツールとの連携 ←**



VScodeからSuperColliderを使う

VScode用のSuperColliderの拡張機能、
これまでもいくつか存在していた (ややこしい…)

- vscode-supercollider: Language support for the SuperCollider language
- language-supercollider: SuperCollider syntax grammars for VSCode
- scvsc: vscode as a supercollider client

しかし、どれも一長一短
更新が止まっていたり、インストールが複雑だったり…

VScodeからSuperColliderを使う

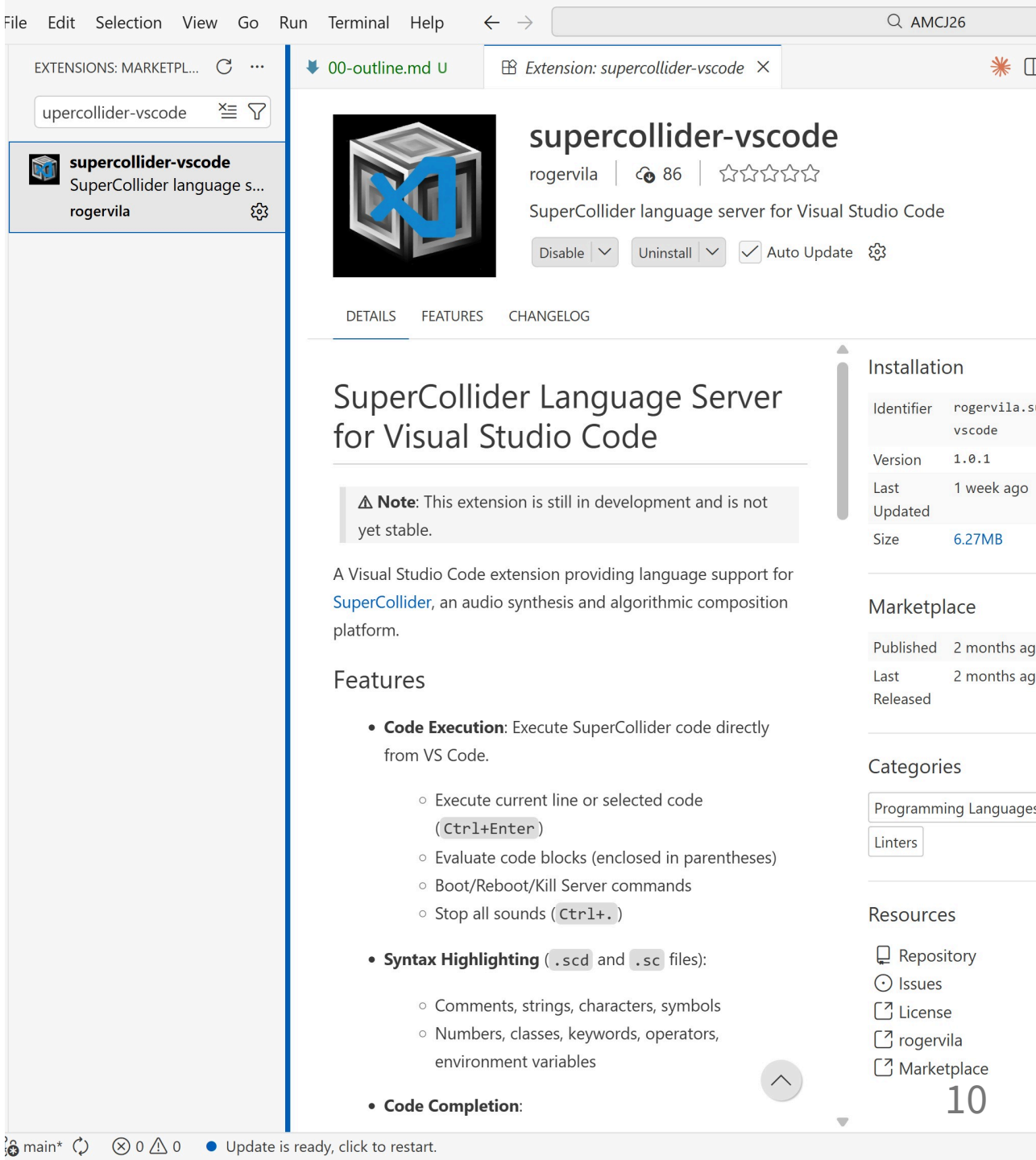
しかし、最近新しい拡張機能が登場! とても便利、かつインストールが (比較的) 簡単!

supercollider-vscode (github)
SuperCollider language server for Visual Studio Code

- コード実行
- シンタックスハイライト
- コード補完
- エディター機能

VScodeからSuperColliderを使う

- VScodeの拡張機能
- “supercollider-vscode” で検索
- インストール



VScodeからSuperColliderを使う

環境設定: 以下のように、SuperColliderの言語であるsclangへのパスを通す必要があります。

macOS: 以下の記述を ~/.zshrc に追加

```
export PATH="$PATH:/Applications/SuperCollider.app/Contents/MacOS"
```

Windows: 環境変数のPATHに以下を追加

```
C:\Program Files\SuperCollider-3.14.1
```

VScodeからSuperColliderを使う

確認: 以下のコマンドがターミナルで実行できるか確認

MacOS: Terminal.appを開いて、以下のコマンドを実行

```
% sclang
```

Windows: PowerShellを開いて、以下のコマンドを実行

```
> sclang
```

AIコード生成を活用しながらSuperColliderをコーディング!!
最強の環境が実現しました!!

00-outline.md U ● | 01-sc-basic.scd ×

01-sc-basic.scd

```
69  // SECTION 3: 音量の変更
70  //
71  /*
72   * SinOsc.ar() の2番目の引数は「位相」、3番目の引数は「振幅（音量）」です。
73   *
74   * SinOsc.ar(freq, phase, mul, add)
75   *   freq  : 周波数（Hz）デフォルト: 440
76   *   phase : 位相（ラジアン）デフォルト: 0.0
77   *   mul   : 振幅（乗数）デフォルト: 1.0
78   *   add   : オフセット デフォルト: 0.0
79   *
80   * 注意: mul=1.0 はフルボリューム。0.0〜1.0 の範囲が適切です。
81   *   1.0を超えるとクリッピング（歪み）が起きる可能性があります。
82   */
83
84  // 音量を0.3に下げる
85  { SinOsc.ar(440, 0.0, 0.3).dup(2) }.play
86
87  // 音量を0.8に上げる
88  { SinOsc.ar(440, 0.0, 0.8).dup(2) }.play
89
90  // キーワード引数を使った書き方（より明瞭）
```

Preview 00-outline.md | 01-sc-basic.scdの内容を教えてくだ...

01-sc-basic.scdの内容を教えてください。

01-sc-basic.scdの内容を教えてください。

c:\Users\tadok\Documents\GitHub\AMCJ26\01-sc-basic.scd

- *Thinking*
- **Read** `c:\Users\tadok\Documents\GitHub\AMCJ26\01-sc-basic.scd`
- 01-sc-basic.scd の内容は以下の通りです（全886行）。

構成

タイトル: Sin波から始めるSuperColliderの基本
12のセクションで構成されたチュートリアルファイルです：

セクション	内容
1	`SinOsc.ar()` — 最初の一音
2	`.dup(2)` / `! 2` — ステレオ再生
3	`mul` 引数 — 音量の変更
4	`freq` 引数 / `.midicps` — 周波数の変更
5	`var` / `~env` — 変数による指定
6	`SinOsc.kr`, `LFNoise1.kr`, `XLine.kr` — コントロール信号による変化（ビブラート、トレモロ等）

ctrl esc to focus or unfocus Claude

Edit automatically | 01-sc-basic.scd

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

Filter (e.g. text, !excludeText, t... | SuperCollider Post Win

```
compile done
localhost : setting clientID to 0.
internal : setting clientID to 0.

-> { SinOsc.ar(440, 0.0, 0.3).dup(2) }.play
Class tree inited in 0.41 seconds
server 'localhost' already booting
```